

Reducing Invoice Rework with AI-Assisted Triage

A fictional Field Notes test article for validating the Flexee manuscript workflow

Jordan Lee (fictional test author)

Word count: 1739

1. The business and the problem

Northstar Industrial Supply is a fictional regional distributor used here only as a software-testing example. It serves manufacturing plants, maintenance teams, and local contractors through a combination of inside sales, field account managers, and an e-commerce portal. The company processes several thousand customer orders and supplier invoices each month. Most transactions move through the accounting system without trouble, but the finance team had a recurring problem: invoices that failed an automated match were being routed into a shared exception queue with very little context.

The exception queue mixed together many different situations. Some invoices had a harmless purchase-order formatting difference. Others had missing receiving records, duplicate freight charges, tax inconsistencies, quantity variances, or genuine pricing disputes. Every exception looked similar when it entered the queue, so experienced analysts had to open the invoice, check the purchase order, inspect receiving notes, read recent email threads, and decide what should happen next. A simple issue might take two minutes, while a complicated one could take twenty. The team spent a large part of each morning sorting work before it could actually resolve work.

The operational impact was larger than the time spent clicking through screens. Urgent supplier invoices were sometimes buried behind low-risk formatting problems. Junior analysts escalated cases that senior staff considered routine because the clues were scattered across systems. Managers also had poor visibility into why the queue grew on certain days. The queue total was easy to measure, but the reasons behind it were not.

Our initial goal was deliberately narrow. We were not trying to let an AI system approve payments, change accounting records, or communicate with suppliers on its own. We wanted to test whether an AI-assisted triage layer could summarize an exception, classify the likely reason, suggest the next internal action, and explain which source evidence supported that suggestion. A human analyst would remain responsible for every accounting decision.

2. What we tried

We started with a four-week pilot using a historical sample of invoice exceptions that had already been resolved by the finance team. For each case, we assembled a small evidence packet containing the invoice text, purchase-order fields, receiving status, supplier notes, and a short extract from the resolution log. We removed bank details and other fields that were not needed for the test.

The first version used a language model to produce three outputs: a one-paragraph case summary, one triage category selected from a controlled list, and a recommended next step. The controlled categories included price variance, quantity variance, missing receipt, suspected duplicate, freight or tax issue, purchase-order reference issue, and needs manual investigation. We required the model to cite the specific fields that influenced its classification. If the model could not find enough evidence, the correct behavior was to say that the case needed

manual investigation rather than invent a confident answer.

We did not begin by connecting the model directly to live finance systems. Instead, we created a small internal review page. An analyst could see the original exception data on the left and the AI-generated summary on the right. The analyst then marked the suggestion as useful, partially useful, or not useful and recorded the action actually taken. This kept the pilot reversible and made it easy to compare the model's suggestions with human judgment.

After the historical test, we ran a limited live trial with a small subset of new exceptions. Cases entered the normal queue first, and the AI-generated triage appeared as additional information rather than as an automated routing rule. Analysts were told to ignore the suggestion whenever the underlying documents disagreed with it. We also created two operational rules. First, any suspected duplicate payment required normal duplicate-payment controls regardless of the model's confidence. Second, no invoice could be approved, rejected, or changed based solely on the generated recommendation.

We measured usefulness rather than model accuracy in isolation. A technically correct label was not valuable if the analyst still had to perform the same amount of investigation. The measures we tracked were time to first useful understanding, percentage of cases where the suggested category matched the final human category, percentage of summaries rated useful, and the number of cases where the suggestion could have caused an incorrect escalation if followed without review.

3. What happened

The clearest benefit was faster orientation. Analysts reported that a concise summary helped them understand the shape of a case before opening every source screen. This was especially useful for exceptions created by small combinations of issues, such as a freight charge plus a receiving delay. In those cases the model often collected the relevant facts into one place more quickly than a person navigating several tabs.

The controlled category list also improved consistency. Before the pilot, two analysts might describe the same issue as "PO problem," "pricing mismatch," or "supplier setup." During the pilot, the categories forced the team to use a more stable vocabulary. That unexpectedly helped managers. Daily queue discussions became more specific because the team could distinguish an increase in missing receipts from an increase in genuine price variances.

The system was most useful on repetitive cases with clear evidence. Missing receipt cases were a good example. When the purchase order matched the invoice but the receiving record was absent, the generated summary usually highlighted that fact and suggested checking whether goods had arrived but had not yet been posted. Analysts still performed the check, but they reached it faster.

The system was less reliable on exceptions whose meaning depended on history outside the evidence packet. A supplier might have a temporary contract arrangement known to the buyer but not represented in the invoice data. A price difference could therefore look like an error when it was actually expected. This reinforced the value of showing citations and uncertainty rather than only a final label.

By the end of the pilot, we did not have evidence to justify autonomous invoice handling, and that had never been the objective. We did have enough evidence to justify continuing an assistive workflow. The most promising operating model was to generate a summary and preliminary category, allow the analyst to confirm or correct it, and use the confirmed category for reporting and queue prioritization. That design kept the human decision at the center while reducing some of the repetitive reading that happened before the decision.

4. What did not work

Our first prompt was too open-ended. We asked the model to explain the problem and recommend what to do next without tightly defining the allowed categories or the boundaries of the recommendation. The result sounded polished but varied too much from case to case. Some responses recommended contacting suppliers even when the evidence packet did not establish that supplier outreach was necessary. Others used different labels for essentially the same problem. The output was readable, but it was not operationally dependable.

The second problem was overconfidence. When evidence was incomplete, the model sometimes chose the most plausible explanation rather than clearly stating that the case was ambiguous. We improved this by adding an explicit "needs manual investigation" category and by requiring evidence citations, but the broader lesson was that uncertainty had to be designed into the workflow. A system that always produces a neat answer can create more risk than one that is allowed to stop.

We also learned that historical resolution notes were not a clean source of truth. Analysts had written them for colleagues, not for machine evaluation. Some notes described the final action but omitted the reasoning. Others used abbreviations that meant different things across teams. In several cases our evaluation initially marked the model wrong, only for a senior analyst to explain that the historical note itself was incomplete. We had to review and normalize part of the test set before drawing conclusions.

Another idea that failed was automatic priority scoring based on the language model alone. We hoped the model could identify urgent exceptions from the narrative context. In practice, urgency was better derived from structured business rules such as payment due date, supplier criticality, dollar value, production impact, and aging. The model could explain those factors, but it was not the right place to invent the priority. We removed that experiment and kept prioritization rule-based.

Finally, the pilot did not eliminate the need for strong process ownership. At first there was a temptation to treat low-quality model output as a prompt-engineering issue. Sometimes it was. Other times the underlying business process was inconsistent. If receiving notes were missing or supplier terms were outdated, better prompting could not repair the source data. The project therefore became partly an AI experiment and partly a process-quality exercise.

The result was less dramatic than the automation story we imagined at the beginning, but it was more useful. The strongest design was not "AI resolves invoice exceptions." It was "AI prepares a concise, evidence-linked first read, while people keep authority over the financial decision." That distinction made the tool easier to trust, easier to audit, and easier to improve.

Practical lessons

Three lessons from the pilot are worth carrying into other business AI projects. First, start with a bounded job that is expensive because of reading and synthesis, not because it requires judgment that the organization is unwilling to delegate. Second, make the system show its evidence and give it a legitimate way to say that it does not know. Third, measure whether the workflow becomes better for the people doing the work, not only whether a model produces labels that score well on a test set.

For Northstar's fictional finance team, the next step would be a controlled production trial with stronger source-data checks, a reviewed evaluation set, and clear monitoring for category drift. Human analysts would continue making the accounting decisions. The AI layer would remain a preparation and triage tool until evidence supported any

broader role.

This article is fictional and was created specifically to test a manuscript submission and editorial-review application. Names, organizations, results, and operational details are illustrative rather than claims about a real company.

AI-use disclosure

This test manuscript was created with AI assistance for drafting, organization, editing, and generation of fictional examples. It is intended solely as test data for the Flexee manuscript workflow and should not be treated as a real practitioner case study.